

PROGRAMAÇÃO MODULAR

SOLID:

*INTERFACE SEGREGATION
DEPENDENCY INVERSION*

PROF. JOÃO CARAM

SOLID PRINCIPLES¹

2

- S** *Single responsibility*
- O** *Open/Closed*
- L** *Liskov substitution*
- I** *Interface segregation*
- D** *Dependency inversion*

¹ - FEATHERS, Michael, a partir de:

MARTIN, Robert C. **Agile software development: principles, patterns, and practices**. Upper Saddle River, N.J.: Prentice Hall, c2003

MARTIN, Robert C; MARTIN, Micah. **Princípios, padrões e práticas ágeis em C#**. Porto Alegre: Bookman, 2011.

SINGLE RESPONSIBILITY PRINCIPLE - SRP

3

“Uma classe deve ter apenas um motivo para mudar.”

- Uma classe deve ter uma única responsabilidade.
 - *Responsabilidade*: motivo para mudança.
- Conceitos relacionados:
 - Coesão, encapsulamento, abstração.

OPEN/CLOSED PRINCIPLE - OCP

4

“Um software deve ser aberto para extensão, mas fechado para modificação.”

- Extensão: classes abstratas e interfaces.
- Conceitos relacionados: herança e polimorfismo.

LSKOV¹ SUBSTITUTION PRINCIPLE - LSP

5

“Objetos que fazem parte de um sistema de software podem ser substituídos por instâncias de seus subtipos sem prejuízo para a correção do sistema.”

- Conceitos relacionados:
 - herança, polimorfismo e abstração.

¹ - LISKOV, Barbara: *Data abstraction and hierarchy*, 1987

6

SEPARAÇÃO DE INTERFACES

INTERFACE SEGREGATION PRINCIPLE - ISP

INTERFACE SEGREGATION PRINCIPLE - ISP

7

“Classes-cliente não devem ser obrigadas a depender de interfaces (ou métodos) os quais não usam.”

- Atenção para herança, composição e associação.
- Usar o serviço x realizar o serviço.

INTERFACE SEGREGATION PRINCIPLE - ISP

8

- Conceitos relacionados:
 - Coesão, abstração e encapsulamento.
- Relação com SRP.

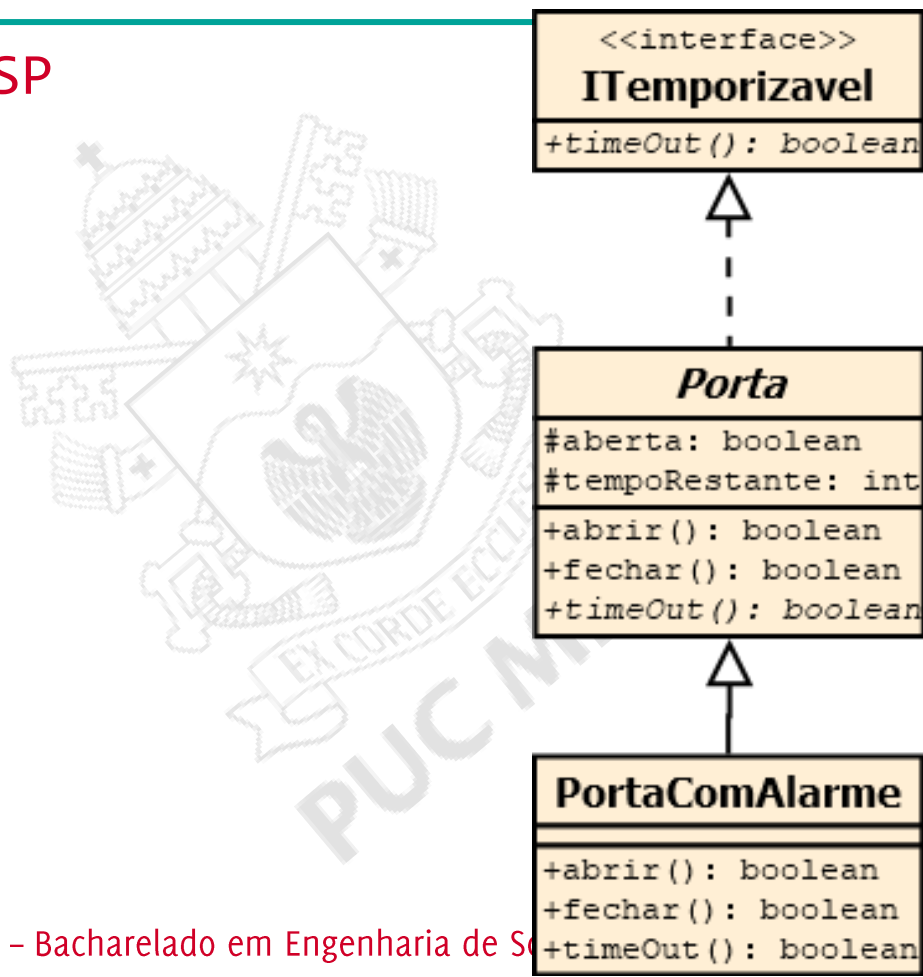
ISP: SEPARANDO INTERFACES

9

- Ex:
 - Uma porta;
 - Um temporizador (*timer*);
 - Uma porta de geladeira que deve disparar um alarme se ficar mais de 20s aberta.

ISP: SEPARANDO INTERFACES

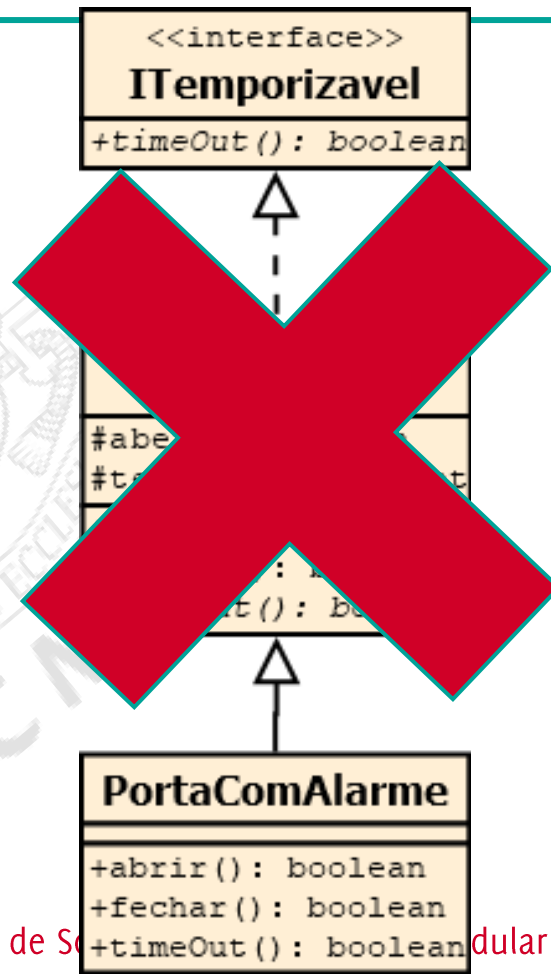
Violação do ISP



ISP: SEPARANDO INTERFACES

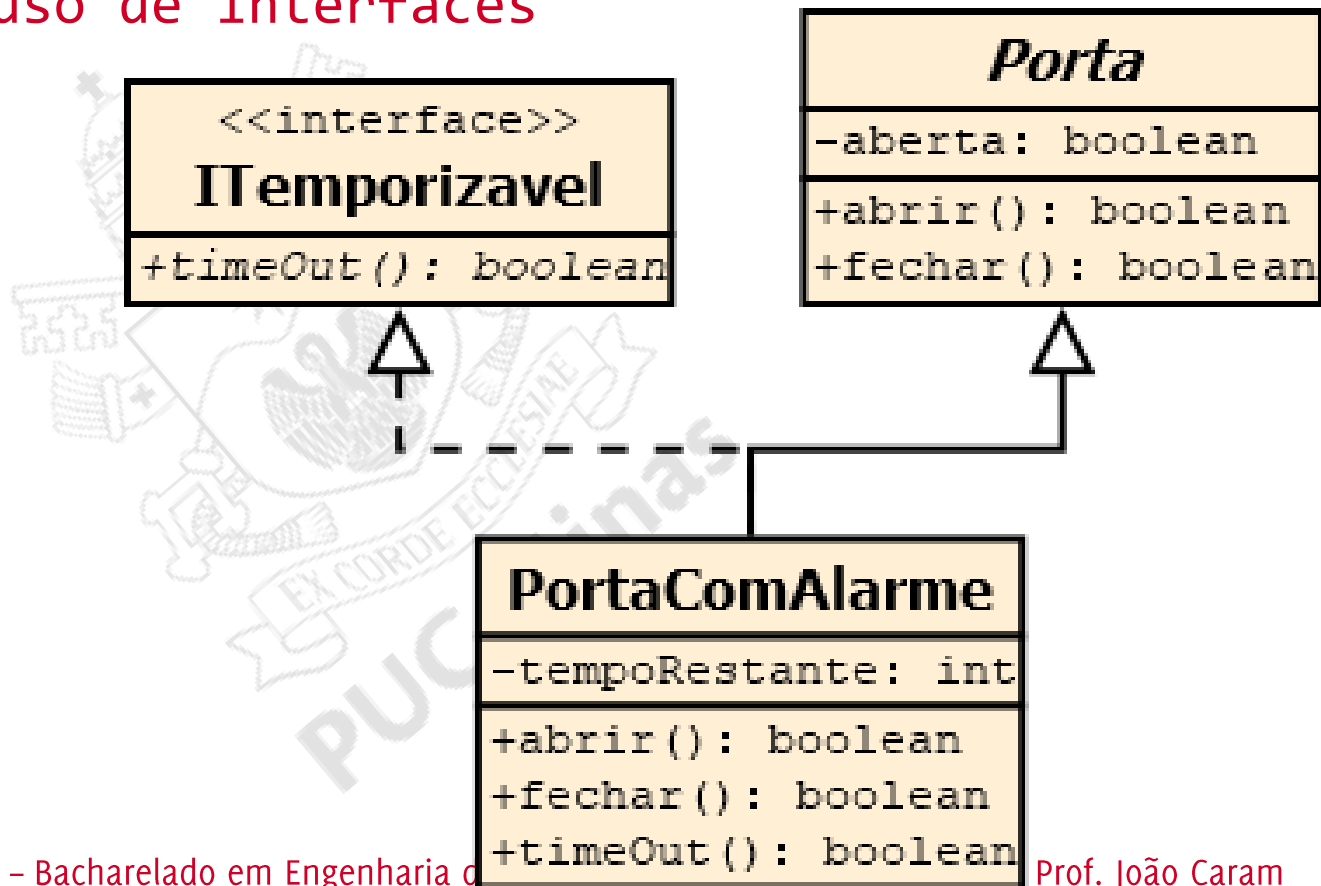
Violação do ISP

Nem toda porta é
temporizável



ISP: SEPARANDO INTERFACES

Melhorando o uso de interfaces



Melhorando o uso de interfaces...

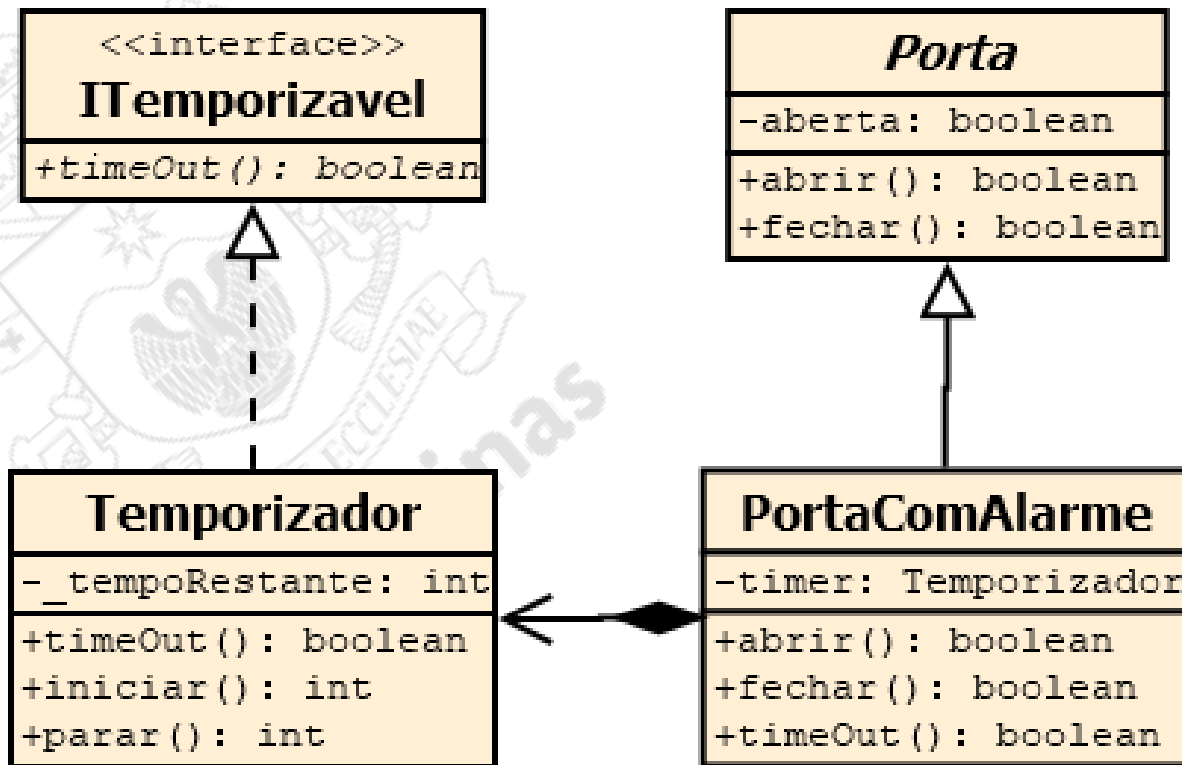
É responsabilidade
de uma porta
marcar o tempo?



ISP: SEPARAÇÃO E COMPOSIÇÃO.

Separando papéis e, além disso, respeitando SRP

PortaComAlarme
utiliza um Timer



15

INVERSÃO DE DEPENDÊNCIA

DEPENDENCY INVERSION PRINCIPLE – DIP

DEPENDENCY INVERSION PRINCIPLE- DIP

16

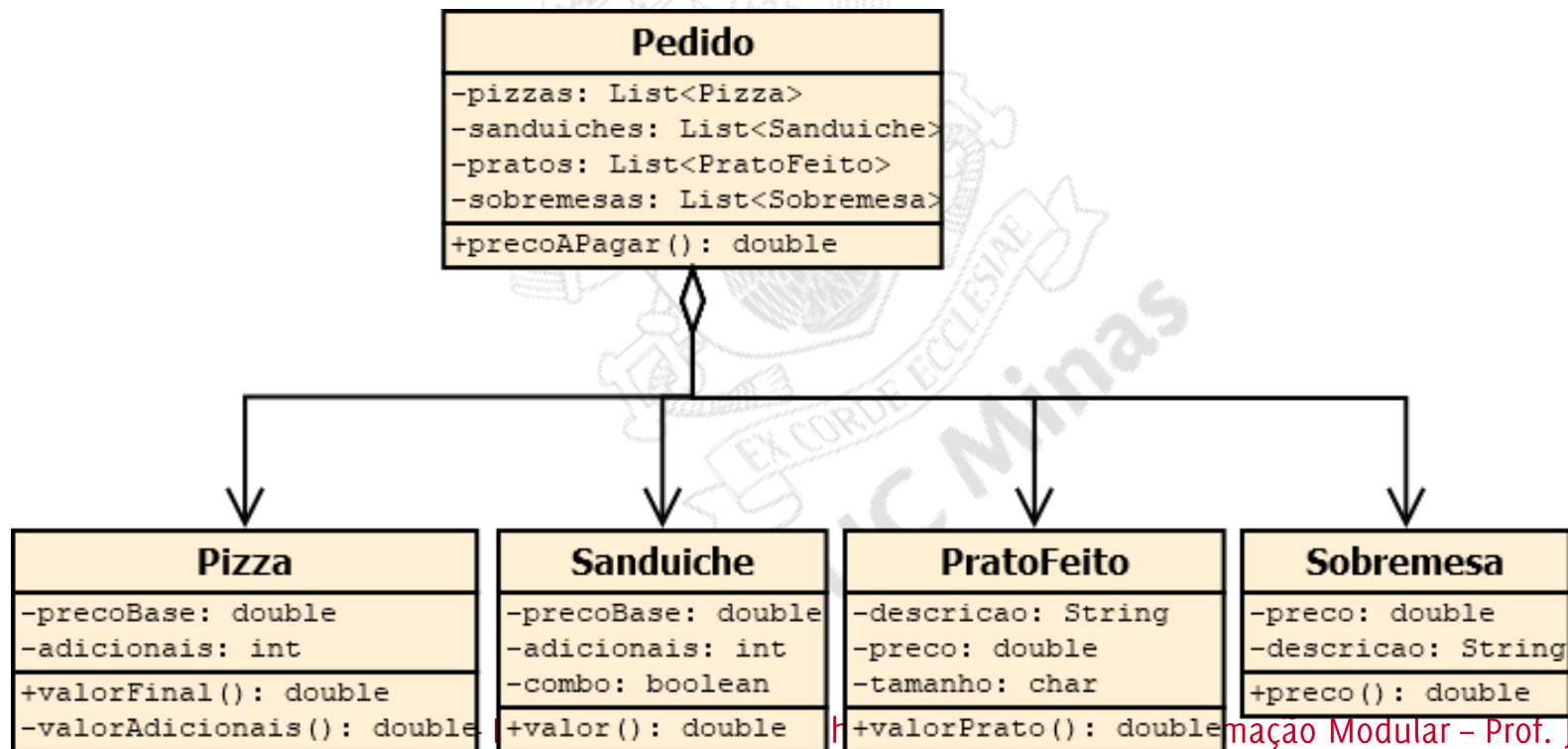
“Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.”

- Conceitos relacionados:
 - Abstração, encapsulamento.

DEPENDENCY INVERSION PRINCIPLE- DIP

Ex: Um pedido tem vários tipos de itens

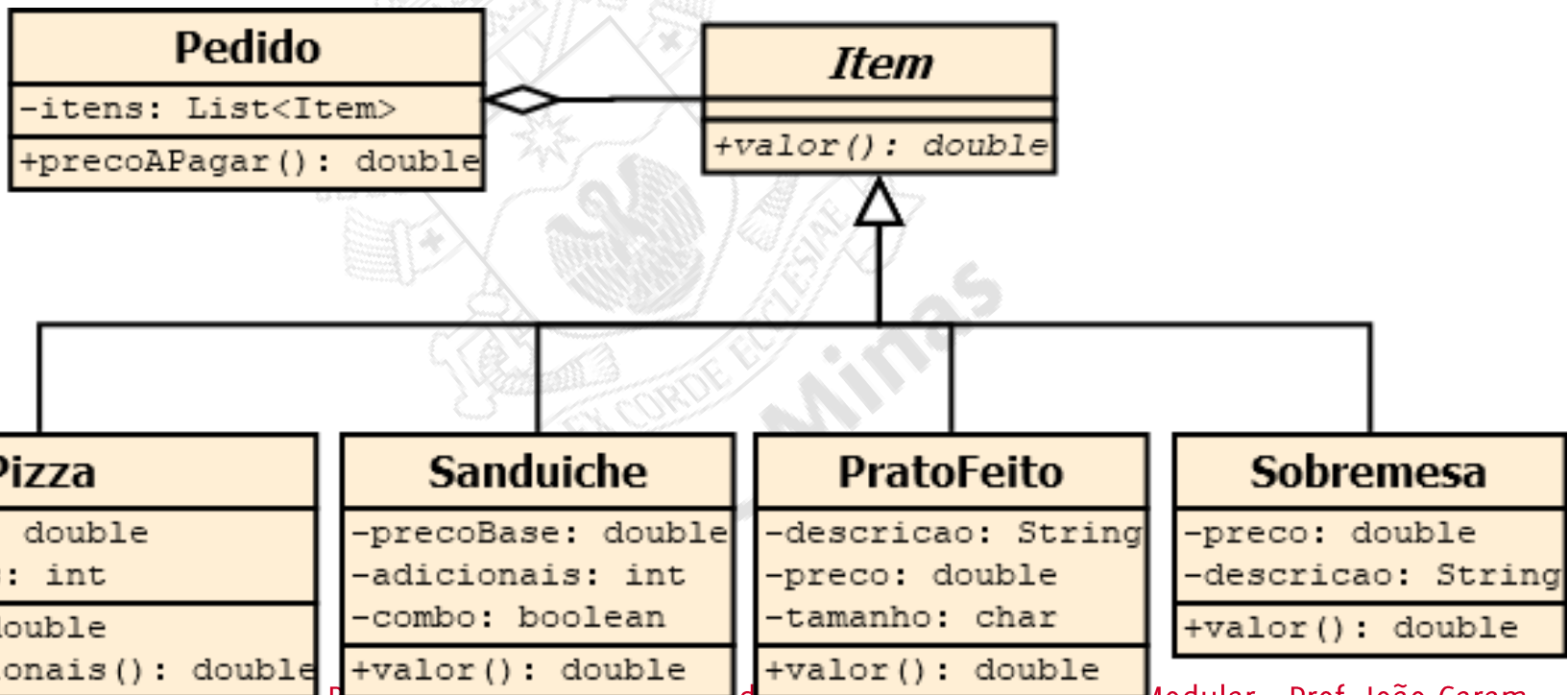
Viola o DIP (pedido depende de cada tipo de item)



DEPENDENCY INVERSION PRINCIPLE- DIP

Ex: Um pedido tem vários tipos de itens

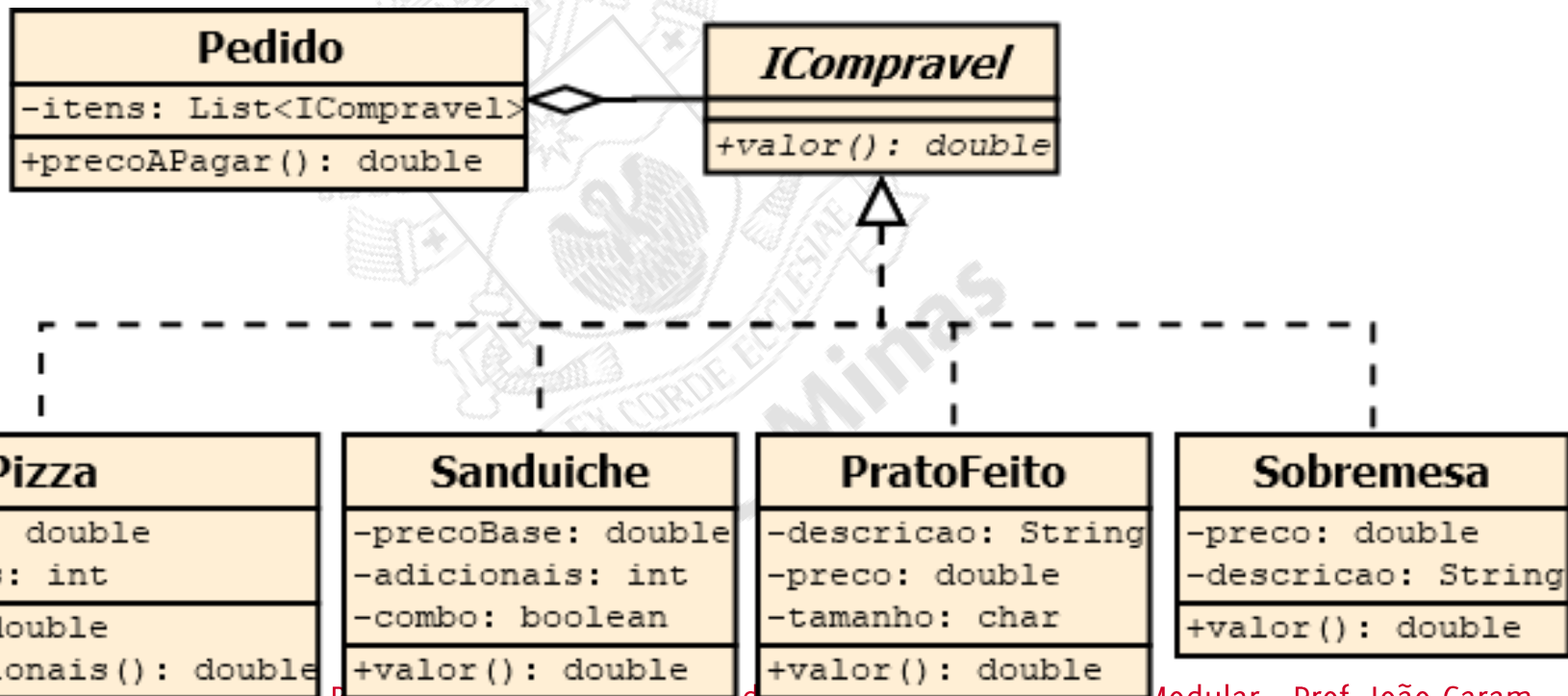
Obedecendo o DIP (todos obedecem a uma abstração)



DEPENDENCY INVERSION PRINCIPLE- DIP

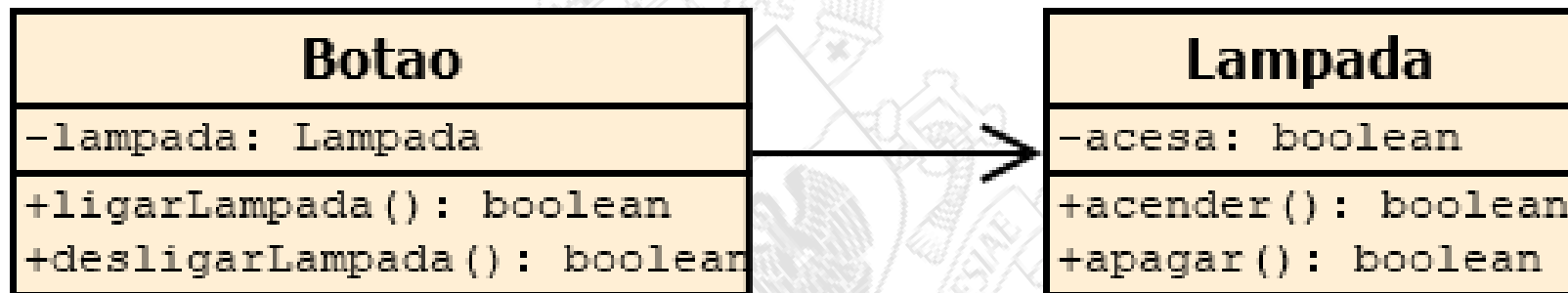
Ex: Um pedido tem vários tipos de itens

Obedecendo o DIP (todos obedecem a uma interface)



DEPENDENCY INVERSION PRINCIPLE- DIP

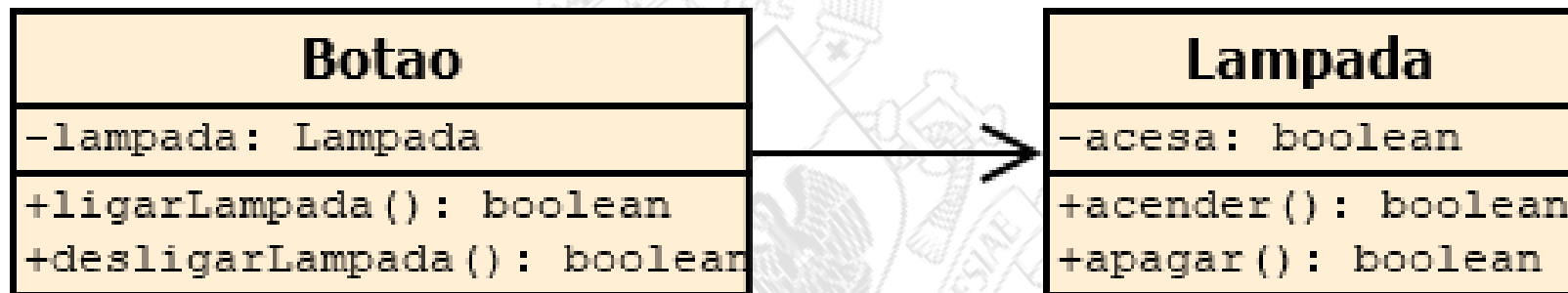
Ex: Um botão para controlar uma lâmpada



DEPENDENCY INVERSION PRINCIPLE- DIP

Ex: Um botão para controlar uma lâmpada

Viola o DIP (botão depende de métodos da lâmpada)

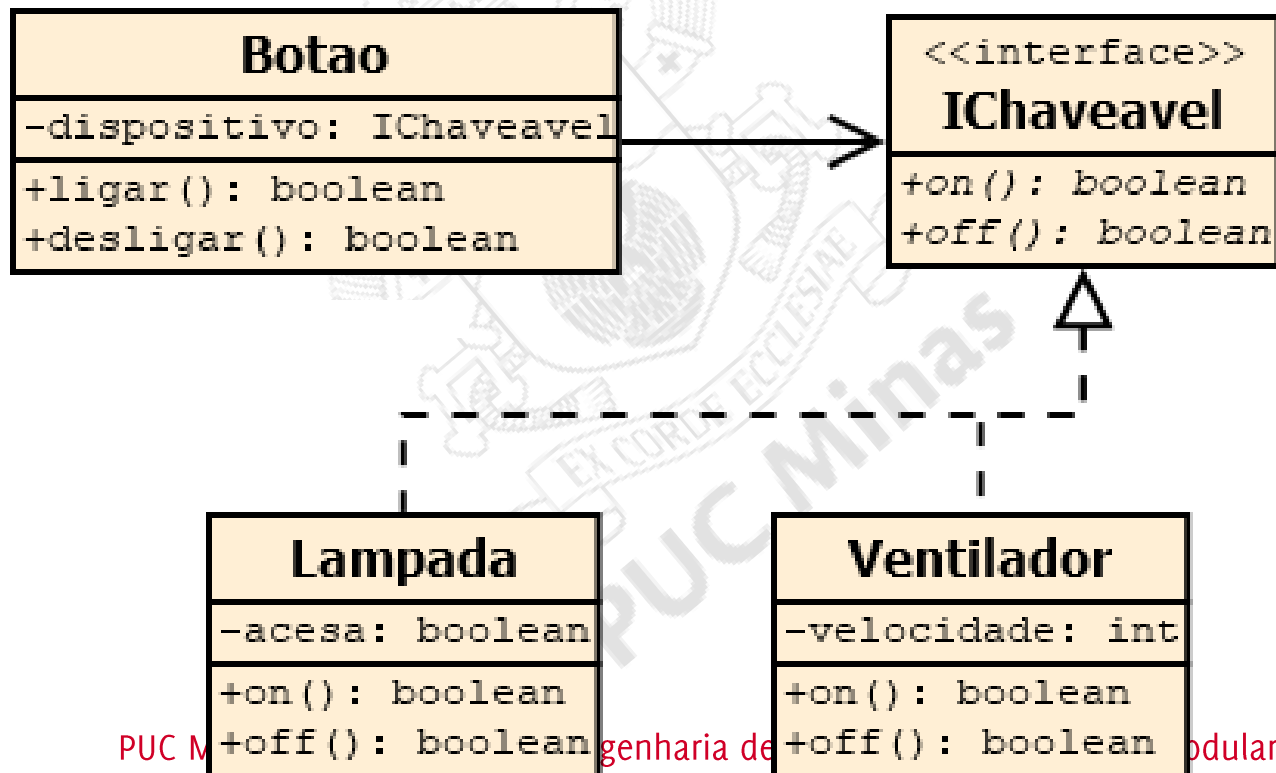


E se “Lampada” mudar?

E se quisermos usar o botão
para controlar outros dispositivos?

DEPENDENCY INVERSION PRINCIPLE- DIP

Ex: Um botão para controlar um chaveável
Obedece ao DIP (ambos usam a interface)



DEPENDENCY INVERSION PRINCIPLE- DIP

23

- Nenhuma variável deveria conter referência para uma classe concreta;
- Nenhuma classe deveria derivar de uma classe concreta;
- Nenhum método deveria sobrescrever métodos implementados em sua classe base.

OBRIGADO.

DÚVIDAS?

PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE